10/Objects

Object-oriented programming (OOP) is a different way to think about your programs. *Objects* are also a way to group variables with related functions. Because you already know how to work with variables and functions, objects simply combine what you've already learned into a more understandable package.

Objects are important, because they break up ideas into smaller building blocks. This mirrors the natural world where, for instance, organs are made of tissue, tissue is made of cells, and so on. Similarly, as your code becomes more complicated, you must think in terms of smaller structures that form more complicated ones. It's easier to write and maintain smaller, understandable pieces of code that work together than it is to write one large piece of code that does everything at once.

Properties and Methods

A software object is a collection of related variables and functions. In the context of objects, a variable is called a *property* (or *instance variable*) and a function is called a *method*. Properties and methods work just like the variables and functions covered in earlier chapters, but we'll use the new terms to emphasize that they are a part of an object. To say it another way, an object combines related data (properties) with related actions and behaviors (methods). The idea is to group together related data with related methods that act on that data.

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

For instance, to model a radio, think about what parameters can be adjusted and the actions that can affect those parameters:

Properties

`volume`, `frequency`, `band`(FM, AM), `power`(on, off)

Methods

`setVolume`, `setFrequency`, `setBand`

Modeling a simple mechanical device is easy compared to modeling an organism like an ant or a person. It's not possible to reduce such complex organisms to a few properties and methods, but it is possible to model enough to create an interesting simulation. *The Sims* video game is a clear example. This game is played by managing the daily activities of simulated people. The characters have enough personality to make a playable, addictive game, but no more. In fact, they have only five personality attributes: neat, outgoing, active, playful, and nice. With the knowledge that it's possible to make a highly simplified model of complex organisms, we could start programming an ant with only a few properties and methods:

Properties

`type`(worker, soldier), `weight`, `length`

Methods

`walk`, `pinch`, `releasePheromones`, `eat`

If you made a list of an ant's properties and methods, you might choose to focus on different aspects of the ant to model. There's no right way to make a model, as long as you make it appropriate for the purpose of your program's goals.

Define a Constructor

To create an object, start by defining a constructor function. A *constructor function* is the specification for an object. Using an architectural analogy, a constructor function is like a blueprint for a house, and the object is like the house itself. Each house made from the blueprint can have variations, and the blueprint is only the specification, not a built structure. For example, one house can be blue and the other red; one house might come with a fireplace and the other without. Likewise with objects, the

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

constructor defines the data types and behaviors, but each object (house) made from a single constructor function (blueprint) has variables (color, fireplace) that are set to different values. To use a more technical term, each object is an *instance* and each instance has its own set of properties and methods.

Before you write a constructor function, we recommend a little planning. Think about what properties and methods your objects should have. Do a little brainstorming to imagine all the possible options and then prioritize and make your best guess about what will work. You'll make changes during the programming process, but it's important to have a good start.

For your properties, select clear names. The properties of an object can hold any type of data. An object can simultaneously hold many booleans, numbers, images, strings, and so on. Keep in mind that one reason to make an object is to group together related data elements. For your methods, select clear names and decide the return values (if any). The methods are used to change the values of the properties and to perform actions based on the properties' values.

For our first constructor function, we'll convert Example 8-9 on page 129 from earlier in the book. We start by making a list of the properties from the example:

```
var x
var y
var diameter
var speed
```

The next step is to figure out what methods might be useful for the object. In looking at the `draw()` function from the example we're adapting, we see two primary components. The position of the shape is updated and drawn to the screen. Let's create two methods for our object, one for each task:

```
function move()
function display()
```

Neither of these methods return a value. Once we've determined the properties and methods the object should have, we'll write our constructor function to assign them to each instance of the object we create (Figure 10-1).

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

```
var red, blue;

function setup() {
  createCanvas(400, 400);
  red = new Train("Red Line", 90);
  blue = new Train("Blue Line", 120);
}

function Train (tempName, tempDistance) {
   this.name = tempName;
   this.distance = tempDistance;
  }
}
```

Assign "Red Line" to the name variable for the red object

Assign 90 to the distance variable for the red object

```
var red, blue;

function setup() {
  createCanvas(400, 400);
  red = new Train("Red Line", 90);
  blue = new Train("Blue Line", 120);
}

function Train (tempName, tempDistance) {
   this.name = tempName;
   this.distance = tempDistance;
  }
}
```

Assign "Blue Line" to the name variable for the blue object

Assign 120 to the distance variable for the blue object

Figure 10-1. *Passing values into the constructor to set the values for an object's properties*

The code inside the constructor function is run once when the object is first created. To create the constructor function, we'll follow three steps:

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

1. Create a function block.
2. Add the properties and assign values to them.
3. Add the methods.

First, we create a function block for our constructor:

```
function JitterBug() {

}
```

Notice that the name `JitterBug` is uppercase. Naming the constructor function with an uppercase letter isn't required, but it is a convention (that we strongly encourage) used to denote that it's a constructor. (The keyword `function`, however, must be lowercase because it's a rule of the programming language.)

Second, we add the properties. JavaScript has a special keyword, `this`, that you can use within the constructor function to refer to the current object. When declaring a property of an object, we leave off the symbol `var`, and instead prepend the variable name with `this.` to indicate that we are assigning a property, a variable of the object. We could declare and assign the `speed` property as follows:

```
function JitterBug() {
  this.speed = 0.5;
}
```

While we are doing this, we have to decide which properties will have their values passed in through the *constructor*. As a rule of thumb, property values that you want to be different for each instance are passed in through the constructor, and the other property values can be defined when they are declared within the constructor, as `speed` is in this case. For the `JitterBug` object, we've decided that the values for `x`, `y`, and `diameter` will be passed in. Each of the values passed in is assigned to a temporary variable that exists only while the code inside the constructor is run. To clarify this, we've added the name `temp` to each of these variables, but they can be named with any terms that you prefer. They are used only to assign the values to the properties that are a part of the object. So we add `tempX`, `tempY`, and `tempDiameter` as parameters for the function, and the properties are declared and assigned as follows:

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

```
function JitterBug(tempX, tempY, tempDiameter) {
  this.x = tempX;
  this.y = tempY;
  this.diameter = tempDiameter;
  this.speed = 0.5; // Same for every instance
}
```

The last step is to add the methods. This is just like writing functions, but here they are contained within the constructor function, and the first line is written a bit differently. Normally, a function to update variables might be written like this:

```
function move() {
  x += random(-speed, speed);
  y += random(-speed, speed);
}
```

Because we want to make this function a method of the object, we again need to use the `this` keyword. The preceding function is converted into a method like this:

```
this.move = function() {
  this.x += random(-this.speed, this.speed);
  this.y += random(-this.speed, this.speed);
};
```

The first line looks a little strange, but the way to interpret it is "create an instance variable (property) called `move`, and assign its value to be this function." Then, any time we refer to properties of the object, we again use `this.`, just as we do when they're initially declared. Putting it together in the constructor looks like this:

```
function JitterBug(tempX, tempY, tempDiameter) {

  this.x = tempX;
  this.y = tempY;
  this.diameter = tempDiameter;
  this.speed = 2.5;

  this.move = function() {
    this.x += random(-this.speed, this.speed);
    this.y += random(-this.speed, this.speed);
  };

  this.display = function() {
    ellipse(this.x, this.y, this.diameter, this.diameter);
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

```
    };

  }
```

Also note the code spacing. Every line within the constructor is indented a few spaces to show that it's inside the block. Within the methods, the code is spaced again to clearly show the hierarchy.

Create Objects

Now that you have defined a constructor function, to use it in a program you must create an object instance from that constructor. There are two steps to create an object:

1. Declare the object variable.
2. Create (initialize) the object with the keyword `new`.

Example 10-1: Make an Object

To make your first object, we'll start by showing how this works within a p5.js sketch and then continue by explaining each part in depth:

```
var bug;

function setup() {
  createCanvas(480, 120);
  background(204);
  // Create object and pass in parameters
  bug = new JitterBug(width/2, height/2, 20);
}

function draw() {
  bug.move();
  bug.display();
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

```
}

// Put a copy of the JitterBug constructor here
```

We declare object variables in the same way as all other variables—the object is declared by writing the keyword `var` followed by a name for the variable:

```
var bug;
```

The second step is to initialize the object with the keyword `new`. It makes space for the object in memory with all its properties and methods. The name of the constructor is written to the right of the `new` keyword, followed by the parameters into the constructor, if any:

```
bug = new JitterBug(width/2, height/2, 20);
```

The three numbers within the parentheses are the parameters that are passed into the `JitterBug` constructor function. The number and order of these parameters must match those of the constructor.

Example 10-2: Make Multiple Objects

In Example 10-1 on page 159, we see something else new: the period (dot) that's used to access the object's methods inside of `draw()`. The dot operator is used to join the name of the object with its properties and methods. It mirrors the way we use `this.` inside the constructor function, but when we refer to it outside the constructor, `this` is replaced by the variable name.

This becomes clearer in this example, where two objects are made from the same constructor. The `jit.move()` function refers to the `move()` method that belongs to the object named `jit`, and `bug.move()` refers to the `move()` method that belongs to the object named `bug`:

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

```
var jit;
var bug;

function setup() {
  createCanvas(480, 120);
  background(204);
  jit = new JitterBug(width * 0.33, height/2, 50);
  bug = new JitterBug(width * 0.66, height/2, 10);
}

function draw() {
  jit.move();
  jit.display();
  bug.move();
  bug.display();
}

// Put a copy of the JitterBug constructor here
```

Now that the constructor exists as its own module of code, any changes will modify the objects made from it. For instance, you could add a property to the `JitterBug` constructor that controls the color, or another that determines its size. These values can be passed in using the constructor or assigned using additional methods, such as `setColor()` or `setSize()`. And because it's a self-contained unit, you can also use the `JitterBug` constructor in another sketch.

Now is a good time to learn about using multiple files in JavaScript. Spreading your code across more than one file makes longer code easier to edit and more manageable in general. A new file is usually created for each constructor, which reinforces the modularity of working with objects and makes the code easy to find.

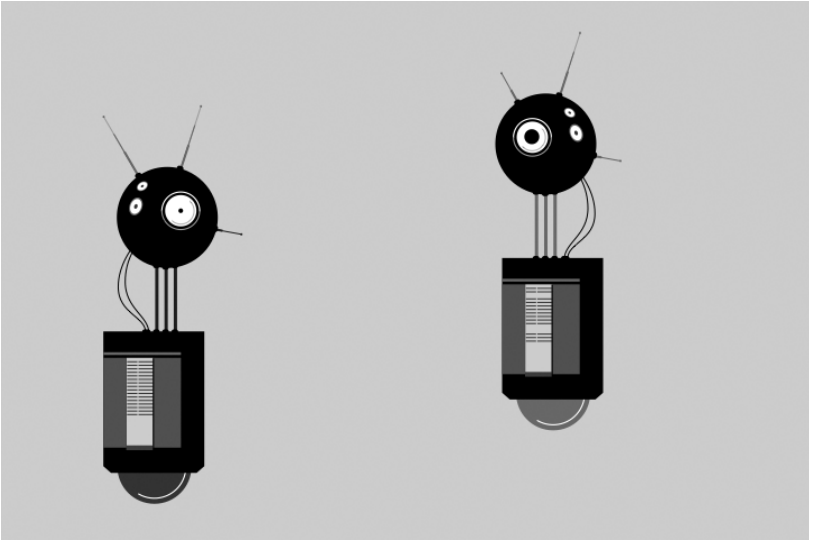
Create a new file in the same folder as your current *sketch.js* file. You can name it anything you like, but it is a good idea to name it *JitterBug.js* for organization. Move the JitterBug constructor function into this new file. Link the *JitterBug.js* file into your HTML file by adding a line in the `HEAD` below the line where you link in the *sketch.js* file:

```
<script type="text/javascript" src="sketch.js"></script>
<script type="text/javascript" src="JitterBug.js"></script>
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

Robot 8: Objects



A software object combines methods (functions) and properties (variables) into one unit. The `Robot` constructor function in this example defines all of the robot objects that will be created from it. Each `Robot` object has its own set of properties to store a position and the illustration that will draw to the screen. Each has methods to update the position and display the illustration.

The parameters for `bot1` and `bot2` in `setup()` define the *x* and *y* coordinates and the *.svg* file that will be used to depict the robot. The `tempX` and `tempY` parameters are passed into the constructor and assigned to the `xpos` and `ypos` properties. The `imgPath` parameter is used to load the related illustration. The objects (`bot1` and `bot2`) draw at their own location and with a different illustration because they each have unique values passed into the objects through their constructors:

```
var img1;
var img2;

var bot1;
var bot2;

function preload() {
```

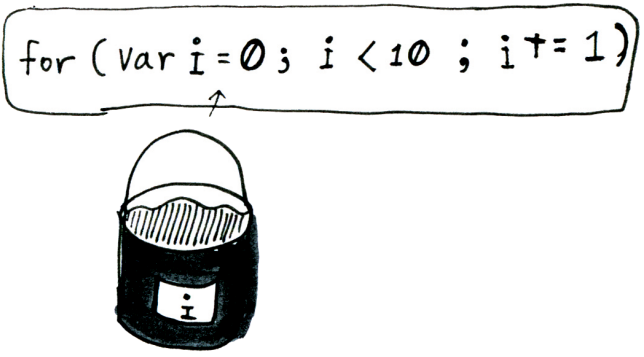
Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing, Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.

```
  img1 = loadImage("robot1.svg");
  img2 = loadImage("robot2.svg");
}

function setup() {
  createCanvas(720, 480);
  bot1 = new Robot(img1, 90, 80);
  bot2 = new Robot(img2, 440, 30);
}

function draw() {
  background(204);

  // Update and display first robot
  bot1.update();
  bot1.display();

  // Update and display second robot
  bot2.update();
  bot2.display();
}

function Robot(img, tempX, tempY) {
  // Set initial values for properties
  this.xpos = tempX;
  this.ypos = tempY;
  this.angle = random(0, TWO_PI);
  this.botImage = img;
  this.yoffset = 0.0;

  // Update the properties
  this.update = function() {
    this.angle += 0.05;
    this.yoffset = sin(this.angle) * 20;
  }

  // Draw the robot to the screen
  this.display = function() {
    image(this.botImage, this.xpos, this.ypos + this.yoffset);
  }
}
```

Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.



Copyright © 2015. Maker Media, Inc. All rights reserved.

McCarthy, Lauren, et al. Getting Started with P5. Js : Making Interactive Graphics in JavaScript and Processing,
Maker Media, Inc, 2015. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/nyulibrary-ebooks/detail.action?docID=4333728.
Created from nyulibrary-ebooks on 2024-07-24 13:01:03.